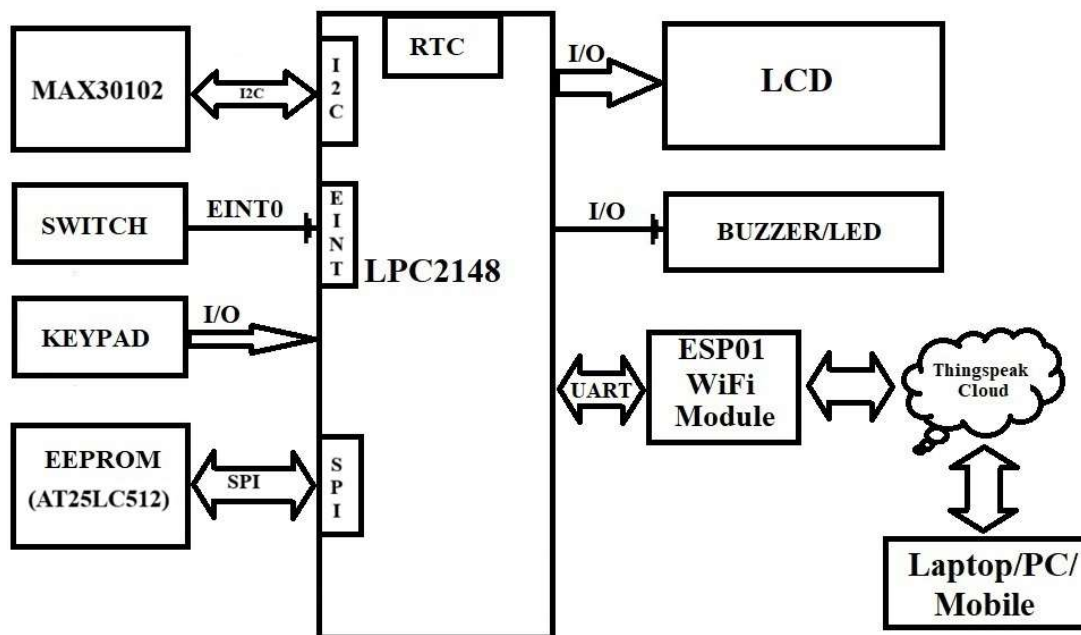


PulseGuard: IoT-Based Heart Rate and Oxygen Monitoring System

AIM: The aim of this project is to design and develop an IoT-based health-monitoring system that continuously measures a person's heart rate and blood oxygen saturation (SpO₂). The system displays the measured values locally and sends them to an IoT platform for remote monitoring. When the heart rate or oxygen level crosses the predefined safe limits, the system generates an alert, enabling timely attention and improving basic health supervision.

Block diagram:



REQUIREMENTS:

HARDWARE REQUIREMENTS:

- LPC2148
- MAX30102
- LCD
- KEYPAD
- SWITCH
- WI-FI MODULE (ESP01)

- AT25LC512
- DB-9 CABLE/USB-UART CONVERTER

SOFTWARE REQUIREMENTS:

- KEIL C Compiler
- PROGRAMMING IN EMBEDDED C
- Flash Magic

Steps to be followed to complete your project:

- Create New Folder in your laptop/PC and save that folder with your project name.
- Then copy what you have done - lcd.c, lcd.h, keypad.c, keypad.h, delay.c, delay.h, spi.c, spi.h, eeprom.c, eeprom.h, uart.c & uart.h, into project folder.
- Individually test each peripheral module before integrating the complete project.
- First check lcd to display character constant, string constant and integer constant.
- Next check keypad peripheral by displaying key values on LCD.
- Next Check UART peripheral by transmitting string constant on hyper terminal.
- Then download the UART interrupt code and check the working on hardware board.
- Next write n bytes into EEPROM and read that n number of bytes from EEPROM and display on LCD. (refer the sample code in LMS)

Note: Use BYTE WRITE and BYTE READ functions

- Need to assign initial set point for temperature. Set point values need to save it in the EEPROM so that each time updated set point will come for the comparison. EEPROM is used to get the updated set points information irrespective of the power failure conditions.
- Then check the external interruption working with the help of any output

device.

- Connect the MAX30102 sensor to the I²C interface of LPC2148. Initialize the LPC2148 I²C peripheral. Verify the I²C address of the MAX30102 module. Read the MAX30102 identification or configuration register to confirm communication.
 - Configure the required MAX30102 registers. Configure the sensor for heart rate and SpO₂ measurement mode.
 - Read the Red LED and IR LED samples from MAX30102 FIFO. Collect sufficient samples before calculating heart rate and SpO₂.
 - Develop or integrate the required algorithm to calculate: Heart Rate in BPM and Blood Oxygen Saturation in %
 - Ask the user to place a finger properly on the MAX30102 sensor.
 - Use averaging or filtering to reduce sudden fluctuations in the measured values.
 - After obtaining stable values, display:
HR: 78 BPM
SpO₂: 97%
- Note: Refer the MAX30102 sample code in LMS.
- Next check the working condition of ESP01 with the help of flash magic terminal. (Which commands are required to use for your project refer to the screen shots under reference data folder)
 - Next connect the ESP01 module to LPC2148. (You can refer to the data sheet of ESP01 to know how to connect)
 - Then develop the driver for ESP01 for LPC2148. By using ESP01 driver, send one constant data to thingspeak cloud.
 - After checking sensors and ESP01 driver then implement the main logic. Inside main initialize all required peripherals.
 - Inside the continuous loop, perform the following operations: Check whether the set-point menu is requested. If requested, process the keypad-

based set-point menu. Check whether a finger is present on the MAX30102. Read the Red and IR sensor samples. Calculate the heart rate and SpO₂ values. Validate and filter the calculated values. Display the latest valid values on the LCD. Compare the heart rate with the low and high set points. Compare SpO₂ with the low set point. Generate an alert when a value crosses its set point. Check the ThingSpeak update timer. Update ThingSpeak when the configured interval expires. Send an abnormal-condition update at the next allowed opportunity. Repeat the complete monitoring process.

- If the user wants to modify the safe-limit values, the user must press the switch connected to EINT0. When the switch is pressed, EINT0 generates an interrupt. The EINT0 ISR must set the `setpoint_menu_request` flag. The main program must detect the flag and temporarily suspend normal LCD updates. The sensor may continue collecting samples in the background, but ThingSpeak transmission must not start while the set-point menu is active.
- Display the following menu on the LCD:
 1. HR LOW LIMIT
 2. HR HIGH LIMIT
 3. SpO₂ LOW LIMIT
 4. UPLOAD TIME
 5. VIEW SETTINGS
 6. EXIT
- Select the required option using the keypad.
- If the user selects option 1, display: HR LOW LIMIT:
- Read the current minimum heart rate set point from EEPROM. Display the current value.

Example: CURRENT: 60

NEW:

- Accept the new value through the keypad. Validate the entered value before saving it. The minimum heart rate set point must be lower than the maximum heart rate set point. If the value is invalid, display:
INVALID VALUE
NOT UPDATED
- If the value is valid, write it into the assigned EEPROM location.
- If the user selects option 2, display: HR HIGH LIMIT:
- Display the current maximum heart rate set point. Accept the new value through the keypad. Validate that the maximum value is greater than the minimum heart rate set point. If the entered value is invalid, retain the old value. If the value is valid, save it in EEPROM. Update the RAM variable.
- If the user selects option 3, display: SpO2 LOW LIMIT:
- Display the current SpO₂ set point. Accept the new value through the keypad. Validate the entered percentage before saving it. If the entered value is invalid, retain the previously stored value. If the value is valid, save it in EEPROM.
- If the user selects option 4, display: UPLOAD TIME:
Display the current update interval in seconds.
Example: CURRENT: 180s
NEW:
- Enter the new upload interval through the keypad. Validate the entered interval before saving it. The interval must be sufficiently long to complete the previous ThingSpeak transaction and comply with the channel update restrictions. If the interval is valid, save it in EEPROM.
- If the user selects option 5, read all set points from EEPROM.
- Display the values one after another:
HR LOW: 60
HR HIGH: 100
SpO2 LOW: 95%

UPLOAD: 180s

- After displaying the saved settings for some time interval, return to the menu.
- If you're getting this output, then your project is completed.

***** ALL THE BEST *****

Note: To make the application more effective and realistic, please review it thoroughly and incorporate all necessary enhancements, improvements, and refinements wherever required.

DIFFERENT IOT APPLICATION PLATFORMS: (FYR)

AWS IoT: Amazon Web Services (AWS) IoT is a cloud-based platform that allows users to connect and manage IoT devices and applications. It provides a wide range of services for data collection, storage, analysis, and visualization.

Microsoft Azure IoT: Microsoft Azure IoT is a comprehensive cloud-based platform that provides tools and services for IoT device management, data analysis, and application development. It supports a wide range of devices and platforms, including Windows and Linux.

Google Cloud IoT: Google Cloud IoT is a cloud-based platform that provides services for IoT device management, data processing, and application development. It supports a wide range of devices and platforms, including Android and iOS.

IBM Watson IoT Platform: IBM Watson IoT Platform is a cloud-based platform that provides services for IoT device management, data analysis, and application development. It supports a wide range of devices and platforms, including Raspberry Pi and Arduino.

ThingSpeak: ThingSpeak is an open-source Internet of Things (IoT) application platform and API (Application Programming Interface) that allows users to collect, analyze and act on data from various sources. It was developed by MathWorks, a leading provider of technical computing software. ThingSpeak provides an easy-to-use interface for users to create and manage IoT devices and applications. It supports a wide range of data sources, including sensors, web services, and social media. The platform also includes features for data visualization, data analysis, and real-time alerts.

Cayenne: Cayenne is a cloud-based IoT platform that provides services for device management, data visualization, and automation. It supports a wide range of devices and platforms, including Arduino and Raspberry Pi.

Kaa IoT Platform: Kaa IoT Platform is an open-source IoT platform that provides tools and services for IoT device management, data processing, and application development. It supports a wide range of devices and platforms, including Android and iOS.

Ubidots: Ubidots is a cloud-based IoT platform that provides services for data collection, visualization, and analysis. It supports a wide range of devices and platforms, including Arduino and Raspberry Pi.

Losant: Losant is a cloud-based IoT platform that provides services for device management, data visualization, and automation. It supports a wide range of devices and platforms, including MQTT and HTTP.

Xively: Xively is a cloud-based IoT platform that provides services for device management, data analytics, and application development. It supports a wide range of devices and platforms, including Arduino and Raspberry Pi.

Blynk: Blynk is a mobile app and cloud-based IoT platform that provides services for device management, data visualization, and control. It supports a wide range of devices and platforms, including Arduino and Raspberry Pi.

Particle: Particle is an IoT platform that provides tools and services for device management, data visualization, and firmware development. It supports a wide range of devices and platforms, including Wi-Fi and cellular.

Thingier.io: Thingier.io is a cloud-based IoT platform that provides services for device management, data collection, and analysis. It supports a wide range of devices and platforms, including ESP8266 and ESP32.

DeviceHive: DeviceHive is an open-source IoT platform that provides services for device management, data analytics, and application development. It supports a wide range of devices and platforms, including Java and .NET.

Mongoose OS: Mongoose OS is an open-source IoT platform that provides services for device management, data analytics, and firmware development. It supports a wide range of devices and platforms, including ESP32 and STM32.

Hologram: Hologram is a cloud-based IoT platform that provides services for device management, data collection, and visualization. It supports a wide range of devices and platforms, including Raspberry Pi and BeagleBone.

ioBroker: ioBroker is an open-source IoT platform that provides services for device management, data visualization, and automation. It supports a wide range of devices and platforms, including Node.js and Docker.

Cayenne: Cayenne is a cloud-based IoT platform that provides services for device management, data visualization, and automation. It supports a wide range of devices and platforms, including Arduino and Raspberry Pi.

Altair SmartWorks: Altair SmartWorks is a cloud-based IoT platform that provides services for device management, data analytics, and application development. It supports a wide range of devices and platforms, including industrial sensors and machines.

Helium: Helium is a decentralized IoT platform that provides services for device management, data collection, and application development. It supports a wide range of devices and platforms.